

Linear Regression with NumPy

This document implements a simple linear regression model using NumPy. Linear regression is a fundamental technique in machine learning and statistics, used to model the relationship between a dependent variable and one or more independent variables. We will begin by describing the dataset in matrix form and then explore the underlying equations and linear algebra related to linear regression. Next, we will evaluate the cost function and demonstrate how to implement gradient descent recursively to minimize it.

1 Representation of the dataset

We create a dataset (x, y) with m rows, $n = 1$ feature x and y as the target variable.

x	y
$x^{(1)}$	$y^{(1)}$
$x^{(2)}$	$y^{(2)}$
$\vdots$	$\vdots$
$x^{(m)}$	$y^{(m)}$

2 Linear Regression model

The model is represented by a function f defined as :

$$f(x) = ax + b \quad (1)$$

In matrix form, we have :

$$F = X\theta \quad (2)$$

where:

$$F = \begin{bmatrix} f(x^{(1)}) \\ f(x^{(2)}) \\ \vdots \\ f(x^{(m)}) \end{bmatrix} ; \dim(F) = (m \times 1) \quad (3)$$

$$X = \begin{bmatrix} x^{(1)} & 1 \\ x^{(2)} & 1 \\ \vdots & \vdots \\ x^{(m)} & 1 \end{bmatrix} ; \dim(X) = (m \times 2) \quad (4)$$

$$\theta = \begin{bmatrix} a \\ b \end{bmatrix} ; \dim(\theta) = (2 \times 1) \quad (5)$$

3 Cost Function

The cost function J is given by the Mean Squared Error (MSE):

$$J(a, b) = \frac{1}{2m} \sum_{i=1}^m \left(ax^{(i)} + b - y^{(i)} \right)^2 \quad (6)$$

In matrix form:

$$J(\theta) = \frac{1}{2m} \sum X^T (X\theta - Y)^2 \quad (7)$$

where:

$$X^T = \begin{bmatrix} x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ 1 & 1 & \dots & 1 \end{bmatrix} \quad (8)$$

and

$$Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix} \quad (9)$$

The gradients are computed as:

$$\frac{\partial J(a, b)}{\partial a} = \frac{1}{2m} \sum_{i=1}^m \frac{\partial}{\partial a} \left(ax^{(i)} + b - y^{(i)} \right)^2 \quad (10)$$

$$= \frac{1}{2m} \sum_{i=1}^m 2x^{(i)} \left(ax^{(i)} + b - y^{(i)} \right) \quad (11)$$

$$= \frac{1}{m} \sum_{i=1}^m \left(ax^{(i)} + b - y^{(i)} \right) x^{(i)} \quad (12)$$

and

$$\frac{\partial J(a, b)}{\partial b} = \frac{1}{2m} \sum_{i=1}^m \frac{\partial}{\partial b} \left(ax^{(i)} + b - y^{(i)} \right)^2 \quad (13)$$

$$= \frac{1}{2m} \sum_{i=1}^m 2 \left(ax^{(i)} + b - y^{(i)} \right) \quad (14)$$

$$= \frac{1}{m} \sum_{i=1}^m \left(ax^{(i)} + b - y^{(i)} \right) \quad (15)$$

In matrix form:

$$\frac{\partial J(\theta)}{\partial \theta} = \begin{bmatrix} \frac{\partial J(a, b)}{\partial a} \\ \frac{\partial J(a, b)}{\partial b} \end{bmatrix} \quad (16)$$

$$= \frac{1}{m} X^T (X\theta - Y) \quad (17)$$

4 Gradient Descent

The gradient descent is an optimization algorithm used to minimize $J(a, b)$ by iteratively updating a and b :

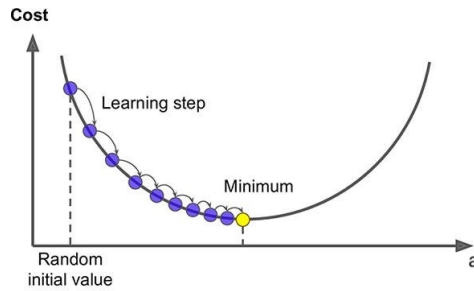


Figure 1: Gradient descent

The optimization of the parameters are defined as this:

$$a_{i+1} = a_i - \alpha \frac{\partial J}{\partial a} \quad (18)$$

$$b_{i+1} = b_i - \alpha \frac{\partial J}{\partial b} \quad (19)$$

where α is the learning rate ($\alpha > 0$). Using the matrices notation, we have :

$$\theta := \theta - \alpha \frac{\partial J}{\partial \theta} \quad (20)$$